

1. INTRODUCTION

Two-wheeled robots are an attractive solution in robotics due to their simplicity and efficiency, but they face significant challenges when it comes to maintaining balance. Unlike four-wheeled robots, two-wheeled robots must constantly adjust their position to stay upright, which becomes even more difficult when external forces such as wind, uneven terrain, or changes in payload weight are introduced. Traditional remote-controlled robots often lack the ability to self-correct, which leads to instability and functional limitations, particularly in dynamic environments.

To overcome these challenges, a self-balancing robot that autonomously maintains stability while allowing for manual control is essential. By integrating Bluetooth technology, this robot would enable precise user control for navigation while adapting to disturbances like weight changes or environmental shifts.

Such a self-balancing robot would not only offer improved stability but also the flexibility needed for various applications, including education, research, and interactive robotics. By utilizing advanced sensors and real-time adaptive control mechanisms, the robot could respond to dynamic changes in its environment, making it suitable for tasks like autonomous delivery, inspection, or assistive devices. Ultimately, the integration of autonomous balance control with manual movement capabilities would push the boundaries of traditional robotics, offering enhanced functionality in real-world settings.

This self-balancing robot would also serve as an ideal platform for further research into adaptive control systems, as its design could be modified or optimized for specific use cases, such as autonomous delivery, inspection tasks, or assistive devices for people with mobility impairments. In summary, the design of a self-balancing robot that can autonomously adjust to external disturbances while still allowing for manual control is a complex yet essential innovation for overcoming the limitations of traditional robots. This technology could pave the way for more resilient, responsive, and versatile robots capable of tackling the

dynamic challenges presented in real-world environments.

PROBLEM STATEMENT

Maintaining stability in a two-wheeled robot while allowing manual control is a complex challenge, especially when external forces or additional weight, such as packages, affect its balance. Traditional remote-controlled robots often lack the ability to self-correct, leading to instability and reduced functionality.

To overcome this issue, a self-balancing robot with automated stability and manual movement control is needed. This system can adapt to external disturbances and weight changes while allowing precise user- controlled navigation via Bluetooth, making it suitable for applications in experimentation, education, and interactive robotics.

OBJECTIVE OF THE PROJECT

- **Implement a PID controller to keep the robot upright.**

Develop PID controller algorithm to process MPU6050 sensor data and dynamically adjust motor speed and direction.

- **Develop a Self-Balancing Mechanism**

Design and implement a self-balancing mechanism using real-time feedback from the MPU6050 sensor and tuning PID algorithm

- **Integrate Wireless Manual Control**

Enable user-directed movement via Bluetooth or WiFi, allowing the robot to navigate in all directions while maintaining balance.

3.WORKING METHODOLOGY AND IMPLEMENTATION

The methodology adopted for the development of the self-balancing robot follows a modular and iterative approach, ensuring the independent validation of each subsystem prior to full integration. The entire process is structured into distinct phases, each with clear objectives and deliverables, as outlined below:

1. Hardware Setup and Assembly

1.1 Component Selection and Integration

The first phase involves selecting and assembling the necessary hardware components. The ESP32 microcontroller is used as the primary processing unit responsible for balancing and movement control. An MPU6050 Inertial Measurement Unit (IMU) is integrated for real-time tilt angle detection. Stepper motors, driven via motor drivers (A4988 or DRV8825), provide precise wheel movement. Additionally, an ESP32-CAM module is used to enable real-time video streaming for remote navigation. A robust power supply system, comprising a battery and voltage regulator, ensures stable operation. The mechanical chassis is assembled with appropriate weight distribution to maintain balance.

1.2 Circuit Design and Connection

The MPU6050 is connected to the ESP32 using the I2C protocol for reliable data transfer. The stepper motors are interfaced with the motor drivers, which are controlled via the ESP32's GPIO pins. The ESP32-CAM is set up independently, operating as a WiFi-based video server.

2. Self-Balancing Algorithm Development

2.1 Sensor Data Processing

The system continuously reads accelerometer and gyroscope data from the MPU6050. To achieve a stable and accurate tilt angle estimate, sensor fusion techniques such as a Complementary Filter or Kalman Filter are applied.

2.2 PID Control Implementation

A Proportional-Integral-Derivative (PID) controller is developed to correct the robot's tilt by dynamically adjusting the speed and direction of the motors. The PID parameters (K_p , K_i , K_d) are carefully tuned to achieve smooth and responsive balancing behavior.

2.3 System Testing and Tuning

Extensive testing is conducted to fine-tune the PID controller. The robot's stability is evaluated on various surfaces and under different conditions, including inclined planes and external disturbances, to ensure robust performance.

3. Manual Control System Integration

3.1 Communication Interface

The ESP32 is configured to receive user commands via Bluetooth or WiFi. A mobile application or a custom remote-control system is developed to transmit directional commands to the robot.

3.2 Motion Control Interface

User commands are processed to initiate forward, backward, and turning motions. These actions are integrated in a manner that does not interfere with the self-balancing functionality.

4. Power Management and Optimization

A suitable battery pack is selected to meet the power demands of the motors, sensors, and microcontrollers. Power efficiency techniques, such as implementing sleep modes and efficient motor control logic, are incorporated to extend battery life.

5. Final Testing and Performance Evaluation

Comprehensive field tests are conducted under various load conditions and environmental disturbances. Key performance metrics, including response time, balance recovery, movement accuracy, and system robustness, are evaluated. The final system is optimized in terms of code execution speed, power consumption, and communication efficiency.

4. Block Diagram:

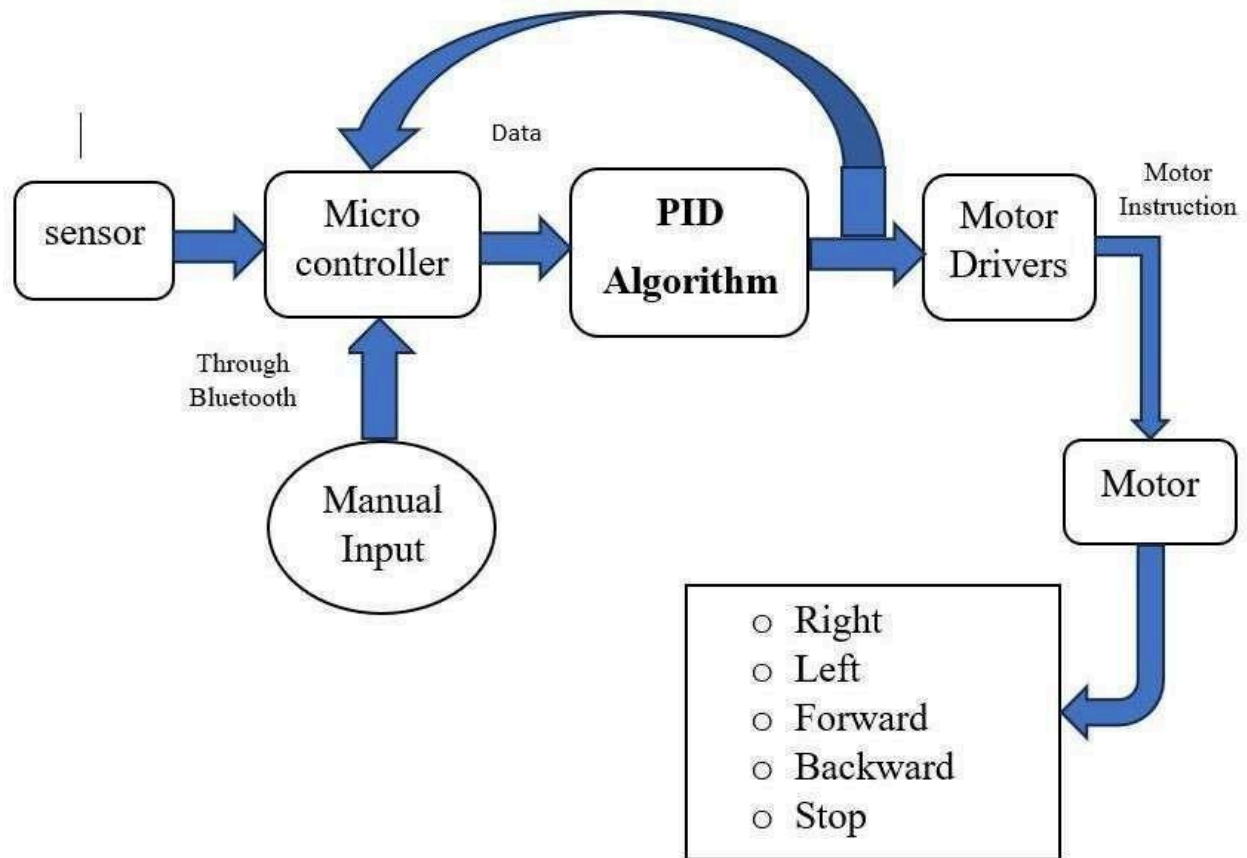


Fig 5.8 Block Diagram of Proposed Methodology

5.Flow Chart :

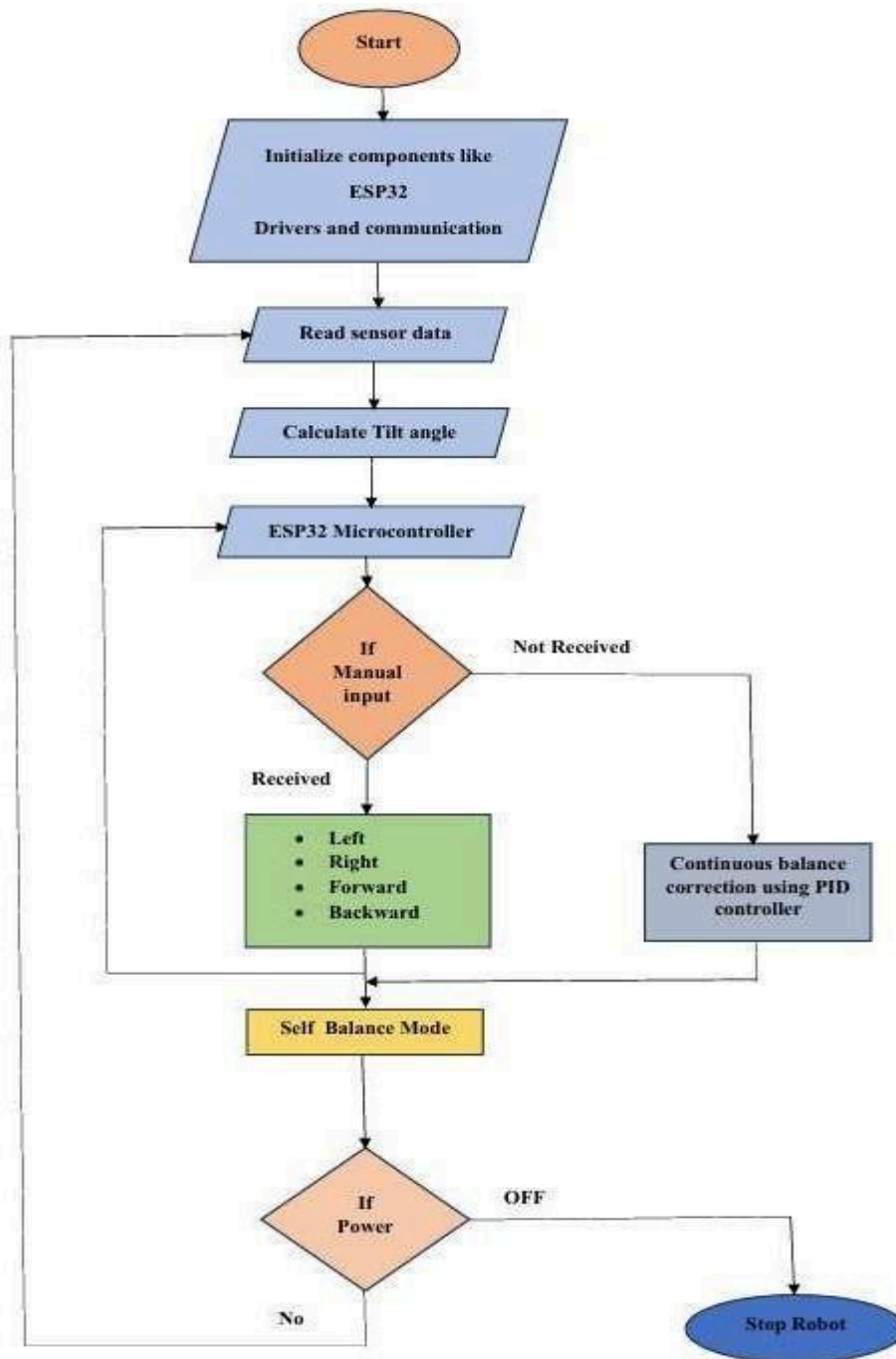


Fig 5.9 Flow Chart of Proposed Methodology

6. Implementation Plan

The self-balancing robot is developed through a systematic and modular approach to ensure a scalable, maintainable, and testable design. This section presents the overall system architecture, hardware and software design considerations, implementation phases, and risk mitigation strategies.

1. System Architecture

The system is divided into three main subsystems: the Balance Control System, the Manual Control Interface, and the Remote Monitoring Unit. These modules work in coordination to allow the robot to maintain upright balance, respond to user commands, and stream live video for remote navigation.

At the core of the system is the ESP32 microcontroller, which processes sensor data in real time and adjusts motor outputs to maintain balance. An MPU6050 Inertial Measurement Unit (IMU) provides accelerometer and gyroscope data, which is processed to calculate the robot's tilt angle. The robot uses stepper motors, driven by A4988 or DRV8825 motor drivers, to perform movement corrections with high precision. The components communicate using the I2C protocol (for MPU6050), GPIO signals (for motor drivers), and WiFi (for remote control).

2. Hardware Design

1.1 Component Selection

- **ESP32:** Chosen as the main controller for its dual-core CPU, built-in WiFi and Bluetooth, and support for real-time tasks
- **MPU6050:** Selected for its integration of a 3-axis gyroscope and 3-axis accelerometer, providing reliable motion and orientation data.
- **Stepper Motors:** Used for their precise control of rotational steps, which are essential for smooth balance correction.
- **Motor Drivers (A4988/DRV8825):** Provide current regulation and microstepping capability to control stepper motors accurately.
- **Power Supply:** A rechargeable Li-ion or Li-Po battery pack is used, regulated by a DC-DC converter to supply required voltages (3.3V and 5V).

- Chassis: Designed with central alignment and symmetry to distribute weight evenly and improve stability.

1.2 Circuit Integration

The MPU6050 is connected to the ESP32 using I2C lines (SCL and SDA). Stepper motors are connected to motor drivers, which are then interfaced with the ESP32 through GPIO pins to control step direction and enable signals. The entire hardware is assembled on a custom-built chassis with secure mounts for mechanical stability.

3. Software Design

The software is developed using a modular approach, enabling independent testing and debugging of each module.

- Sensor Module: Captures raw accelerometer and gyroscope data from the MPU6050 at regular intervals.
- Filter Module: Applies a Complementary Filter or Kalman Filter to fuse sensor data and calculate the tilt angle accurately.
- Control Module: Implements a PID (Proportional-Integral-Derivative) controller that computes motor output based on the tilt error.
- Motor Driver Module: Sends step and direction signals to the motor drivers based on the control signal.
- Communication Module: Listens for remote commands from a mobile or PC interface via Bluetooth or WiFi.

The code is written to ensure real-time response, with the main control loop executing at high frequency (typically 100–200 Hz) to maintain balance.

4. Implementation Plan

The implementation is divided into detailed phases that cover the entire development lifecycle:

Phase 1: Hardware Setup and Wiring

- Design the mechanical chassis and ensure symmetrical weight distribution.
- Mount the ESP32, MPU6050, stepper motors, and motor drivers securely.
- Connect MPU6050 to ESP32 via I2C interface.
- Interface stepper motors through A4988/DRV8825 drivers using ESP32 GPIOs.
- Set up power circuitry with voltage regulators and battery connectors.

Phase 2: Sensor Data Acquisition and Filtering

- Read raw data from MPU6050 (accelerometer and gyroscope).
- Apply a complementary filter or Kalman filter to compute a stable tilt angle.
- Validate tilt angle accuracy by comparing with theoretical tilt.

Phase 3: PID Control Implementation

- Design the PID controller for real-time correction of tilt errors.
- Implement logic to adjust motor direction and speed based on PID output.
- Conduct multiple tuning sessions for PID parameters (K_p , K_i , K_d).
- Observe behavior and optimize responsiveness while avoiding oscillation.

Phase 4: Manual Control Integration

- Enable WiFi or Bluetooth on ESP32 for remote control.
- Develop a mobile or desktop interface to send directional commands.
- Integrate received commands with the PID control loop.
- Ensure user commands like forward, backward, and turning do not destabilize the robot.

Phase 5: Power Management System

- Select a battery with sufficient capacity based on total current draw.
- Design and test voltage regulation circuitry (buck converter).
- Monitor battery voltage and add indicators or safety cutoffs.
- Optimize code to reduce power consumption during idle or standby

Phase 6: System Testing and Performance Optimization

- Test the robot on various surfaces (smooth, inclined, uneven).
- Apply push and load disturbances to evaluate recovery behavior.
- Measure balance stability, motor response time, and overall accuracy.
- Test communication reliability and video streaming latency.
- Finalize PID values and optimize control loop timing.

5. Risk Management and Mitigation

- Sensor Drift: Minimized using complementary or Kalman filters for reliable tilt estimation.
- Power Fluctuations: Mitigated using stable DC-DC regulators and battery level monitoring.
- Motor Overheating: Controlled by limiting driver current and adding heat sinks.
- Communication Delays: Reduced by optimizing WiFi packet size, loop timing, and limiting unnecessary background processes.

This design and implementation plan forms a structured foundation for the development of the self-balancing robot. It ensures each phase is clearly defined, technically justified, and aligned with the goals of real-time operation, precise motion control, and wireless functionality. This document serves as a core component of the final report submitted for academic evaluation.

REQUIRED COMPONENTS



Fig 5.1 100*2 wheels for robots

The 100*2 wheels for robots are 100mm dual-wheel units designed for stability, durability, and smooth movement. Made from high-quality rubber or TPU, they offer excellent grip and traction, making them ideal for autonomous robots, AGVs, and terrain-adaptive platforms. Their dual-wheel design enhances load distribution and reduces skidding, ensuring efficient movement. Compatible with DC, stepper, and servo motors, these wheels provide reliable performance for precision-driven robotic applications.



Fig 5.2 NEMA 17 stepper motor

The NEMA 17 stepper motor is a widely used, high-precision motor ideal for robotics, 3D printers, CNC machines, and automation systems. It has a 1.7-inch (43.2mm) frame size, offering a balance between torque and compactness. With 1.8° step angle (200 steps per revolution), it provides accurate and controlled motion. The motor operates on bipolar or unipolar configurations and is compatible with common motor drivers. Known for its high torque, precise positioning, and reliability, the NEMA 17 is a preferred choice for applications requiring controlled rotational movement and repeatability.



Fig 5.3 LiPo (Lithium Polymer) battery

A LiPo (Lithium Polymer) battery is a lightweight, high-energy-density rechargeable battery commonly used in robotics, drones, RC vehicles, and portable electronics. It offers a high discharge rate, making it ideal for applications requiring bursts of power. LiPo batteries come in various voltages (measured in cells or "S" rating, e.g., 3S = 11.1V) and capacities (measured in mAh), allowing customization based on power needs. They require careful handling to prevent overcharging, deep discharging, or physical damage. With proper usage and maintenance, LiPo batteries provide efficient power delivery, longer runtimes, and lightweight energy storage for high-performance applications.

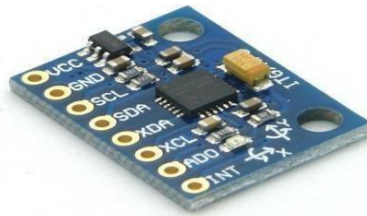


Fig 5.4 MPU6050

The MPU6050 is a 6-axis motion tracking sensor that combines a 3-axis gyroscope and a 3-axis accelerometer in a single compact module. It is widely used in robotics, drones, gaming controllers, and motion analysis for detecting orientation, acceleration, and angular velocity. The sensor communicates via I2C protocol, making it easy to interface with microcontrollers like Arduino and Raspberry Pi. With built-in Digital Motion Processing (DMP), it can perform complex calculations internally, reducing the processing load on the microcontroller. The MPU6050 is ideal for applications requiring precise motion tracking, stabilization, and gesture recognition.



Fig 5.5 A4988 stepper motor driver

The A4988 stepper motor driver is a microstepping motor driver used to control bipolar stepper motors with precision. It supports full, half, quarter, eighth, and sixteenth-step resolutions, allowing for smooth and accurate motion control. The driver operates on 8V to 35V input voltage and can deliver up to 2A per coil with proper cooling. It features built-in current limiting, thermal shutdown, and crossover current protection, ensuring safe and efficient operation. The A4988 communicates via simple step and direction signals, making it easy to use with microcontrollers like Arduino and Raspberry Pi. Commonly used in 3D printers, CNC machines, and robotics, it provides precise and reliable motor control.



Fig 5.6 ESP32

The ESP32 is a powerful Wi-Fi + Bluetooth microcontroller developed by Espressif Systems, designed for IoT, automation, and embedded applications. It features a dual-core Xtensa LX6 processor running at up to 240 MHz, with 520 KB SRAM, and 4MB to 16MB flash memory. It supports Wi-Fi 802.11 b/g/n, Bluetooth 4.2 (Classic + BLE), and has multiple GPIOs, ADCs, DACs, I²C, SPI, UART, PWM, and capacitive touch inputs. With low-power modes, it is ideal for battery-operated projects. It operates at 3.3V, is programmable via Arduino IDE, ESP-IDF, or MicroPython, and is widely used in smart home systems, industrial automation, robotics, and IoT application.

COMPONENTS CONNECTIONS :

1. ESP32 Pin Connections

ESP32 GPIO Pin	Connected To	Purpose
GPIO 18	A4988 STEP (Motor 1)	Step signal for Motor 1
GPIO 19	A4988 DIR (Motor 1)	Direction signal for Motor 1
GPIO 25	A4988 STEP (Motor 2)	Step signal for Motor 2
GPIO 26	A4988 DIR (Motor 2)	Direction signal for Motor 2
GPIO 21 (default SDA)	MPU6050 SDA	I2C data line
GPIO 22 (default SCL)	MPU6050 SCL	I2C clock line

2. A4988 Stepper Driver (x2 for Motor 1 and Motor 2)

Each A4988 driver module needs the following connections:

A4988 Pin	Connects To	Description
STEP	ESP32 GPIO (18 or 25)	Step pulse input
DIR	ESP32 GPIO (19 or 26)	Motor direction input
EN (Optional)	GND (active low)	Enables the driver
VDD	3.3V or 5V from ESP32	Logic voltage
GND	ESP32 GND	Common ground
VMOT	External Power (12V recommended)	Motor voltage
GND (Power)	Power supply ground	Ground for motor side
2B, 2A, 1A, 1B	Stepper Motor coils	Connect motor wires

PID Algorithm:

Complementary Filter(angle): A simplified form of sensor fusion is used:

`angle = alpha * (angle + gyroRate * dt) + (1 - alpha) * accelAngle;`

`alpha(0.98)` weights the gyroscope-derived angle (integrated over time `dt`) heavily, as it's good for sudden movements.

`(1 - alpha)(0.02)` weights the accelerometer-derived angle, which slowly corrects the gyroscope's drift.

This filter provides a robust and relatively clean estimate of the robot's pitch angle by combining the strengths of both sensors.

Accelerometer Angle Calculation (in degrees):

`accelAngle = atan2(accY, accZ) * 180.0 / PI;`

This gives the **angle of tilt** based on acceleration along the Y and Z axes.

Formula:

$$\theta_{\text{accel}} = \tan^{-1} \left(\frac{a_Y}{a_Z} \right) \times \left(\frac{180}{\pi} \right)$$

1. Gyroscope Angular Rate Integration:

`gyroRate = gyroX / 131.0;`

`angle += gyroRate * dt;`

This adds angular velocity over time to estimate how much the angle has changed. `gyroX` is in raw sensor units.

Dividing by 131.0 converts it to degrees per second (as per MPU6050 specs in $\pm 250^\circ/\text{s}$ mode).

`dt` is time in seconds between measurements.

2. Complementary Filter for Sensor Fusion:

angle = alpha * (angle + gyroRate * dt) + (1 - alpha) * accelAngle;

This combines both gyroscope and accelerometer values using a **complementary filter**.

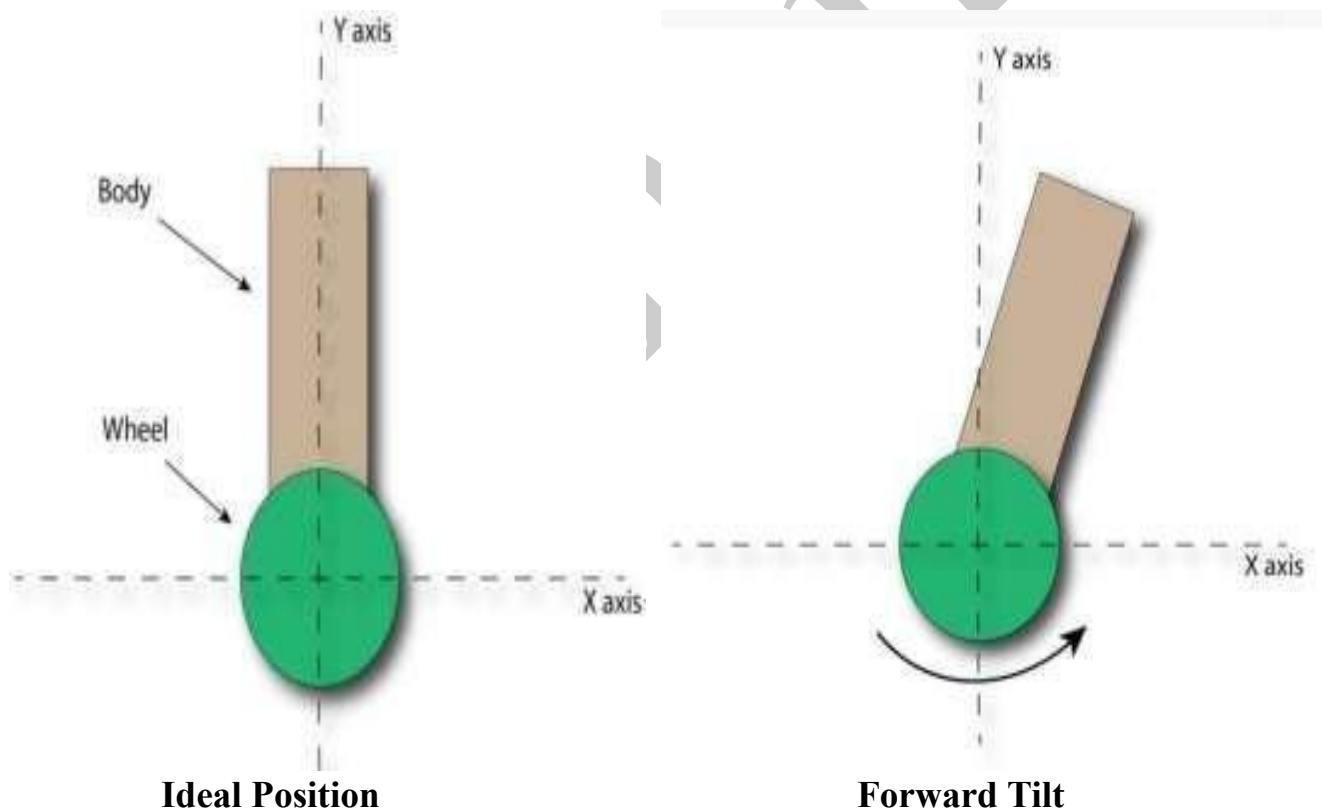
Final Tilt Angle Formula:

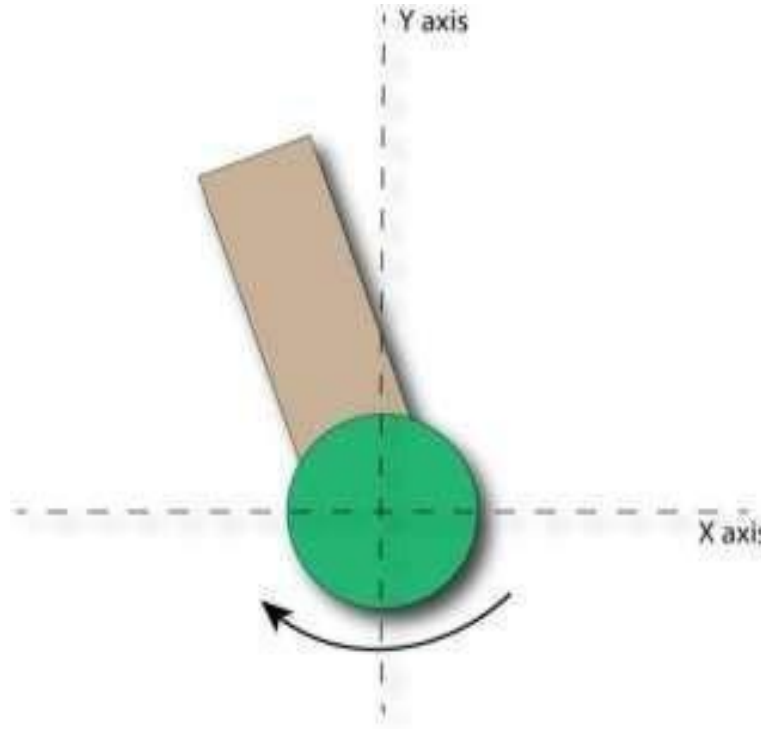
$$\theta = \alpha \left(\theta_{\text{prev}} + \frac{\omega_{\text{gyro}}}{131} \cdot \Delta t \right) + (1 - \alpha) \cdot \theta_{\text{accel}}$$

Where:

- $\alpha = 0.98$ (trust mostly the gyro)
- ω_{gyro} is `gyroX` in raw value
- Δt is time difference in seconds
- $\theta_{\text{accel}} = \tan^{-1} \left(\frac{a_Y}{a_Z} \right) \cdot \frac{180}{\pi}$

DIRECTIONS OF TILT ANGLE





Backward Tilt

Ideal Position : In its ideal state, the robot's body is perfectly upright, with a zero angle between its Y-axis and the robot's body. This represents the target equilibrium state the control system constantly strives to maintain.

Forward Tilt Correction : When the robot's body tilts forward, the MPU6050 gyro sensor immediately detects this angular deviation. This data is relayed to the esp32, which, after performing PID calculations, commands the stepper motors to run forward. This forward motion generates a corrective torque that minimizes the forward tilt angle, bringing it back towards zero.

Backward Tilt Correction : Conversely, if the robot tilts backward, the same closed-loop process occurs. The MPU6050 detects the backward tilt, and the esp32 commands the motors to rotate backward. This action generates a torque that corrects the backward tilt angle, restoring the robot to its upright equilibrium.

Conclusion:

This project successfully demonstrates the development of a **two-wheeled self-balancing robot** using the ESP32, MPU6050 IMU sensor, and A4988 stepper motor drivers, with the AccelStepper library providing smooth motor control.

The ESP32 was selected for its powerful dual-core processor, integrated Bluetooth, and strong community support. The robot uses real-time angle feedback from the MPU6050 sensor, processed with a complementary filter, and a PID controller to maintain balance. The system also features Bluetooth communication that enables real-time PID tuning, status feedback, and manual control inputs using apps like Serial Bluetooth Terminal or Bluino Controller.

The robot is able to stand upright and balance for up to 1–2 minutes with careful tuning of the PID parameters and target angle compensation. While manual directional control (forward, backward, left, right) over Bluetooth is functional, it has not yet been fully tuned, and may cause instability if used abruptly—especially when the robot is already near the edge of its balance threshold.

Key outcomes:

Stable balancing behavior using PID control.

Complementary filtering for clean tilt angle estimation.

Bluetooth-based tuning and control interface.

Manual control works but requires smoother transition logic and tuning to avoid balance loss. PID and target angle values saved to ESP32 flash for persistence across reboots.

This project demonstrates an important integration of embedded systems, control theory, and real-time sensor fusion.